

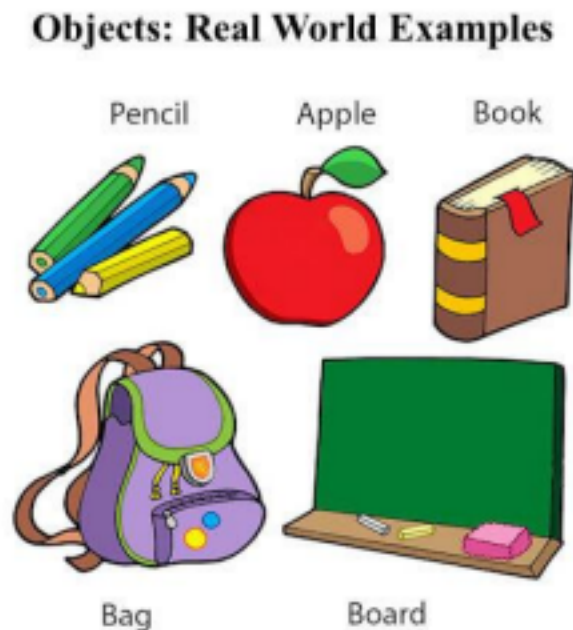
# Objects and Classes in Java

What is an object in Java

An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible). The example of an intangible object is the banking system.

An object has three characteristics:

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.



For Example, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.

**An object is an instance of a class.** A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

## Object Definitions:

- An object is *a real-world entity*.
- An object is *a runtime entity*.
- The object is *an entity which has state and behavior*.
- The object is *an instance of a class*.

## What is a class in Java

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interface**

Syntax to declare a class:

```
class <class_name>
{
    field;
    method;
}
```

## Instance variable in Java

A variable which is created inside the class but outside the method is known as an instance variable. Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.

## Method in Java

In Java, a method is like a function which is used to expose the behavior of an object. Advantage of Method

- Code Reusability
- Code Optimization
- 

## new keyword in Java

- The new keyword is used to allocate memory at runtime. All objects get memory in

Heap memory area.

## Object and Class Example: main within the class

Here, we are creating a main() method inside the class.

```
class Student
{
    int id;
    String name;
    public static void main(String args[])
    {
        Student s1=new Student();
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

## Object and Class Example: main outside the class

we are having main() method in another class.

We can have multiple classes in different Java files or single Java file. If you define multiple classes in a single Java source file, it is a good idea to save the file name with the class name which has main() method.

```
class Student
{
    int id;
    String name;
}
class TestStudent1
{
    public static void main(String args[])
    {
        Student s1=new Student();
        System.out.println(s1.id);
        System.out.println(s1.name);
    }
}
```

## 3 Ways to initialize object

There are 3 ways to initialize object in Java.

1. By reference variable
2. By method

### 3. By constructor

## 1. Initialization through reference

Initializing an object means storing data into the object. Let's see a simple example where we are going to initialize the object through a reference variable.

```
class Student
{
    int id;
    String name;
}
class TestStudent2
{
    public static void main(String args[])
    {
        Student s1=new Student();
        s1.id=101;
        s1.name="Abhijeet";
        System.out.println(s1.id+" "+s1.name);//printing members with a white space
    }
}
```

**We can also create multiple objects and store information in it through reference variable.**

```
class Student
{
    int id;
    String name;
}
class TestStudent3
{
    public static void main(String args[])
    {
        //Creating objects
        Student s1=new Student();
        Student s2=new Student();
        //Initializing objects
        s1.id=101;
        s1.name="Ram";

        s2.id=102;
        s2.name="Sham";
    }
}
```

```

        //Printing data
        System.out.println(s1.id+" "+s1.name);
        System.out.println(s2.id+" "+s2.name);
    }
}

```

## 2) Initialization through method

In this example, we are creating the two objects of Student class and initializing the value to these objects by invoking the insertRecord method. Here, we are displaying the state (data) of the objects by invoking the displayInformation() method.

### **class Student**

```

{
    int rollno;
    String name;
    void insertRecord(int r, String n)
    {
        rollno=r;
        name=n;
    }
    void displayInformation()
    {
        System.out.println(rollno+" "+name);
    }
}

```

### **class TestStudent4**

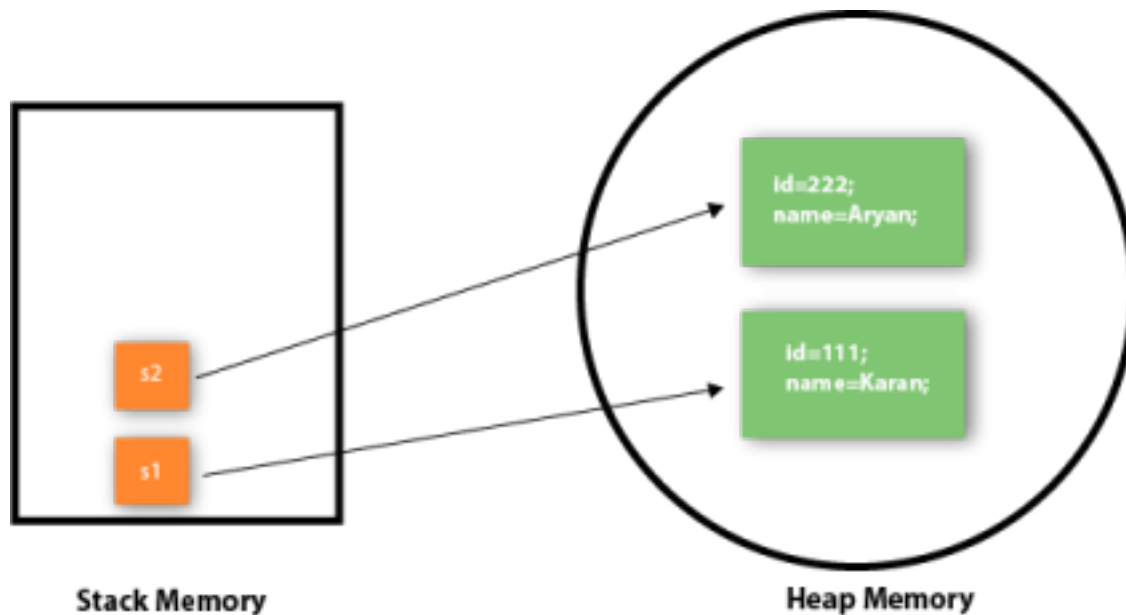
```

{
    public static void main(String args[])
    {
        Student s1=new Student();
        Student s2=new Student();

        s1.insertRecord(111,"Karan");
        s2.insertRecord(222,"Aryan");

        s1.displayInformation();
        s2.displayInformation();
    }
}

```



As you can see in the above figure, object gets the memory in heap memory area. The reference variable refers to the object allocated in the heap memory area. Here, s1 and s2 both are reference variables that refer to the objects allocated in memory.

### Object and Class Example: Employee

Let's see an example where we are maintaining records of employees.

```
class Employee
{
    int id;
    String name;
    float salary;
    void insert(int i, String n, float s)
    {
        id=i;
        name=n;
        salary=s;
    }
    void display()
    {
        System.out.println(id+" "+name+" "+salary);
    }
}

public class TestEmployee
{
    public static void main(String[] args)
    {
        Employee e1=new Employee();
        Employee e2=new Employee();
    }
}
```

```

        Employee e3=new Employee();

        e1.insert(101,"ajeet",45000);
        e2.insert(102,"irfan",25000);
        e3.insert(103,"nakul",55000);

        e1.display();
        e2.display();
        e3.display();
    }
}

```

## Object and Class Example: Rectangle

There is given another example that maintains the records of Rectangle class.

```

class Rectangle
{
    int length;
    int width;
    void insert(int l, int w)
    {
        length=l;
        width=w;
    }
    void calculateArea()
    {
        System.out.println(length*width);
    }
}
class TestRectangle1
{
    public static void main(String args[])
    {
        Rectangle r1=new Rectangle();
        Rectangle r2=new Rectangle();

        r1.insert(11,5);
        r2.insert(3,15);

        r1.calculateArea();
        r2.calculateArea();
    }
}

```

## What are the different ways to create an object in Java?

There are many ways to create an object in java. They are:

- By new keyword
- By newInstance() method
- By clone() method
- By deserialization
- By factory method etc.

## Anonymous object

Anonymous simply means nameless. An object which has no reference is known as an anonymous object. It can be used at the time of object creation only.

If you have to use an object only once, an anonymous object is a good approach. For example: **new** Calculation();//anonymous object

Calling method through a reference:

```
Calculation c=new Calculation();
c.fact(5);
```

## Calling method through an anonymous object

```
new Calculation().fact(5);
```

```
class Calculation
{
    void fact(int n)
    {
        int fact=1;
        for(int i=1;i<=n;i++)
        {
            fact=fact*i;
        }
        System.out.println("factorial is "+fact);
    }
    public static void main(String args[])
    {
        new Calculation().fact(5);//calling method with anonymous object
    }
}
```

## Real World Example: Account

```
class Account
{
    int acc_no;
    String name;
    float amount;
    //Method to initialize object
    void insert(int a,String n,float amt)
```



```

    {
        acc_no=a;
        name=n;
        amount=amt;
    }
    //deposit method
    void deposit(float amt)
    {
        amount=amount+amt;
        System.out.println(amt+" deposited");
    }
    //withdraw method
    void withdraw(float amt)
    {
        if(amount<amt)
        {
            System.out.println("Insufficient Balance");
        }
        else
        {
            amount=amount-amt;
            System.out.println(amt+" withdrawn");
        }
    }
    //method to check the balance of the account
    void checkBalance()
    {
        System.out.println("Balance is: "+amount);
    }
    //method to display the values of an object
    void display()
    {
        System.out.println(acc_no+" "+name+"
"+amount); }
}
//Creating a test class to deposit and withdraw amount
class TestAccount
{
    public static void main(String[] args)

```

```
{  
    Account a1=new Account();  
    a1.insert(832345,"Ankit",1000);  
  
    a1.display();  
    a1.checkBalance();  
  
    a1.deposit(40000);  
    a1.checkBalance();  
  
    a1.withdraw(15000);  
    a1.checkBalance();  
}  
}
```